

ETAPA 2 / 2 - DESENVOLVIMENTO DA COLETA DE DADOS E COMUNICAÇÃO TCP/IP -

EQUIPE ALPHA TECH

Nossa equipe estava enfrentando problemas, pois não conseguimos entender o que o nosso cliente havia nos instruído de fazer como ele queria fazer, precisávamos rever o que estava acontecendo, a equipe entregou uma pequena parte do que nos foi proposto, as pesquisas foram consultadas e relidas para a documentação necessária.

Enfrentamos dificuldades em relação ao Arduino, nós não estávamos conseguindo conectar a Internet para realizar os outros testes, nosso programador depois de muita luta, conseguiu fazer a conexão TCP/IP com o Arduino. Entretanto, não foi possível terminar as outras partes do nosso trabalho que nos foi encarregado de fazer, muitas dificuldades vieram no caminho, e não conseguimos terminar da melhor forma o nosso relatório de forma completa e organizada.

Houve certas discussões no meio da equipe, nossos integrantes discutiram sobre a responsabilidade da criação do banco de dados, os dois membros ficaram com falta de comunicação sobre o envio de dados para o líder do grupo que está escrevendo esta documentação agora mesmo, o líder conseguiu resolver o problema rapidamente, um está agora estruturando o banco de dados e enviando as informações para o seu companheiro que faz a programação e integração do SQL para o Python.

Atualmente, nossa segunda parte para a coleta de dados está sendo feita e trabalhada agora, uma membra do nosso trabalho acabou de terminar o nosso banco de dados que já está sendo programado dentro do Python para testes e verificações dos códigos, identificando todo o procedimento da programação.

Como o bando foi estruturado?

A gente montou o banco usando tabelas que conversam. A principal é a de sensores, e as outras são de Leitura, Alertas, etc. são ligadas a ela pelo id do sensor. Usamos chaves primárias para não repetir dados e chaves estrangeiras para garantir que toda leitura ou alerta venha de um sensor que realmente existe.

Como os dados serão armazenados?

Tudo o que o sensor capta vai direto para as tabelas. Guardamos os números tipo temperatura e umidade em colunas de números e as mensagens em colunas de texto. O legal é que a gente configurou o banco para salvar o dia e a hora toda vez que entrar um dado novo, assim a gente não perde o histórico.

Como será feita a consulta das informações?

A consulta das informações será feita através de comandos sql no banco de dados. O sistema vai poder usar consultas como select para ver os dados das tabelas, como leituras, alertas, eventos e logs.

Como o banco de dados se conecta ao sistema?

O banco de dados se conecta ao sistema através de uma API em Python. O ESP8266 envia dados para a API via WI-FI, e a API faz a comunicação com o SQL Server para poder guardar as informações.

Explicando o que foi feito nas Tabelas

```
1 CREATE TABLE sensores (  
2     id_sensor INT IDENTITY(1,1) PRIMARY KEY,  
3     nome VARCHAR(50),  
4     localizacao VARCHAR(100),  
5     status VARCHAR(20)  
6 );
```

A tabela de Sensores, que é a base de tudo, onde cadastramos o nome e onde fica cada aparelho.

```

1  CREATE TABLE leituras (
2      id_leitura INT IDENTITY(1,1) PRIMARY KEY,
3      id_sensor INTEGER REFERENCES sensores(id_sensor),
4      temperatura DECIMAL(5,2),
5      umidade INTEGER,
6      rotacao INTEGER,
7      estado VARCHAR(20),
8      data_hora DATETIME DEFAULT GETDATE()
9  );

```

A tabela de Leituras, que é onde guardamos temperaturas, umidade e rotação que o sensor manda.

```

1  CREATE TABLE eventos (
2      id_evento INT PRIMARY KEY,
3      descricao TEXT,
4      nivel VARCHAR(20),
5      data_hora DATETIME DEFAULT GETDATE()
6  );

```

A tabela Eventos serve para marcar coisas importantes que acontecem no sistema.

```

1  CREATE TABLE alertas (
2      id_alerta INT PRIMARY KEY,
3      id_sensor INT REFERENCES sensores(id_sensor),
4      mensagem TEXT,
5      status VARCHAR(20),
6      data_hora DATETIME DEFAULT GETDATE ()
7  );

```

A tabela Alertas foi criada para guardar os alertas gerados pelo sistema. Nela ficam registradas informações importantes, quando acontece algo fora do normal, como exemplo a temperatura alta ou falha em algum sensor.

```
1 CREATE TABLE logs (  
2     id_log INT PRIMARY KEY,  
3     origem VARCHAR(50),  
4     mensagem TEXT,  
5     data_hora DATETIME DEFAULT GETDATE ()  
6 );
```

A tabela logs foi criada para registrar acontecimentos e erros do sistema.

PESQUISA DE INTEGRAÇÃO COM GOOGLE SHEETS

O objetivo dessa pesquisa foi entender como os dados do nosso banco de dados podem ser enviados e sincronizados com o Google Sheets para facilitar a organização das informações do sistema.

Como ocorre a integração entre banco de dados e Google Sheets?

A integração acontece usando uma API em Python. O ESP8266 envia os dados pela internet, a API recebe essas informações, salva no banco de dados e depois manda tudo para o Google Sheets usando a API do Google.

Quais tecnologias podem ser utilizadas?

Podem ser usadas tecnologias como Python, SQL Server, Google Sheets, JSON, entre outros. Elas ajudam na comunicação entre o ESP8266, o banco de dados e o Google Sheets.

Como uma API pode participar desse processo?

A API ajuda porque ela manda as informações de um lugar para o outro de maneira automática, aí as pessoas conseguem mexer no documento juntas, sem precisar ficar salvando toda hora e atualizando.

Quais dificuldades podem existir?

As dificuldades podem aparecer quando a internet fica lenta, erro na API, também quando duas pessoas escrevem a mesma coisa e buga tudo ou perde alguma parte

do texto.

INVESTIGAÇÃO

Google Sheets API

A Google Sheets ajuda a conectar nosso sistema com as planilhas do Google. Com ela, os dados do banco podem ser enviados automaticamente para o Google Sheets sem precisar colocar manualmente.

Integração via Python

A integração será feita usando Python, porque ele facilita a comunicação entre o banco de dados e o Google Sheets. O python vai receber os dados, salvar no banco e depois enviar para a planilha automaticamente.

Webhooks

Webhooks são notificações automáticas enviadas quando acontece algum evento. Eles permitem que um sistema avise outro em tempo real. Um exemplo é: alguém alterar uma planilha, o sistema recebe um aviso e o outro programa atualiza os dados automaticamente.

Bibliotecas de integração

Algumas bibliotecas ajudam a conectar o Python com o Google Sheets de um jeito mais simples. Alguns exemplos de biblioteca: gspread facilita mexer nas planilhas, google-api-python-client é uma biblioteca oficial das APIs Google, pandas ajudam a organizar e analisar dados.

Métodos de autenticação

A autenticação será feita usando uma chave do Google para permitir que o Python consiga acessar a planilha do Google Sheets com segurança.

Tecnologia Escolhida

A tecnologia escolhida para a integração foi Python com Google Sheets API, porque facilita a integração entre o banco de dados e a planilha, além de automatizar o envio

dos dados.

Biblioteca Utilizada

A biblioteca que estamos utilizando é a: Google Sheets API

Método de Autenticação

Nosso código de autenticação está funcionando como uma base de login + conexão com banco de dados MySQL usando Python.

Fluxo da Informação

ESP8266 → API Python → SQL Server → Google Sheets

DESCREVENDO

Como os dados sairão do ESP8266?

Os dados saem do ESP8266 pelo WI-FI. Ele pega as informações dos sensores e envia tudo em formato JSON pela rede usando TCP/IP.

Como chegarão ao banco?

Os dados chegam em uma API feita em Python, que recebe, organiza e salva no banco de dados SQL Server.

Como serão enviados ao Google Sheets?

Depois de salvar no banco de dados, o sistema utilizará a Google Sheets API para mandar automaticamente as informações para uma planilha online. O Python vai fazer a conexão com API, a autenticação usando Service Account e o envio das informações para as células da planilha.

Quais ferramentas serão utilizadas?

Utilizaremos o ESP8266, Python, MySQL, Google Sheets API e as bibliotecas: gspread, google-api-python-client e mysql.connector.

PROBLEMAS ENCONTRADOS

Durante o projeto, encontramos alguns problemas que podem acontecer no sistema:

Como problemas de conexão, às vezes o Wi-Fi pode demorar muito ou até não conectar, dependendo de quantas pessoas estão usando a rede ou pode até ser a qualidade da rede também.

Erros na comunicação entre o Python e o banco de dados.

O código que o professor passou não está executando, fica dando erro.

Estamos também enfrentando dificuldades no Google Sheets API porque está meio lento para mexer.

E também não estamos conseguindo usar os botões das placas, mesmo conferindo o código várias vezes.

INTEGRAÇÃO INICIAL COM API

O objetivo é fazer primeiro o envio de dados para testar se o sistema está funcionando.

APIs REST

É o que faz a comunicação entre o ESP8266, o Python e o banco de dados pela internet.

Métodos de envio

A gente usa o método POST para enviar os dados em JSON do ESP8266 para a API.

Simulação e testes

Fazemos testes enviando os dados para ver se estão chegando na API, indo para o banco e funcionando tudo certo sem erro.

TESTES E VALIDAÇÕES

O objetivo é garantir que o sistema estivesse funcionando corretamente e organizado.

Nós verificamos se:

- O banco de dados estava armazenando os dados
- Organização no Banco MySQL
- Testamos as portas de entradas e saídas do arduino com o Buzzer, para dar aqueles “alarmes”

Resultado

Os testes mostraram que o sistema está funcionando, mas ainda pode precisar de alguns ajustes até ficar mais estável e organizado.

REGISTRO OBRIGATÓRIO

Mudanças Realizadas

A gente reorganizou as tarefas e ajustou algumas partes do banco de dados e do envio de informações.

Melhorias Aplicadas

Melhoramos a organização do código, o JSON e a comunicação entre a API e o banco de dados para evitar erros.

Ajustes de Lógica

Ajustamos o fluxo para garantir que os dados saiam do ESP8266, passem pela API e cheguem no banco corretamente.

Ajustes do Banco

Melhoramos as tabelas, usamos melhor as chaves primárias e estrangeiras e deixamos o banco mais organizado.

EVIDÊNCIAS OBRIGATÓRIAS

As evidências do projeto são registros que mostram tudo o que a gente fez e testou no sistema. Elas servem para comprovar o desenvolvimento e o funcionamento do nosso projeto.

Precisamos juntar:

- Prints do banco de dados com os dados sendo salvos
- Estrutura das tabelas do SQL Server
- Pesquisas feitas sobre API, Google Sheets e integração
- Fluxogramas explicando o funcionamento do sistema
- Prints da comunicação entre o ESP8266 e API
- Fotos da montagem do projeto
- Logs e erros encontrados nos testes

ANÁLISE CRÍTICA

O banco escolhido atende o projeto? Por quê?

Sim, o MySQL atende o projeto porque ele consegue guardar os dados dos sensores de forma organizada e facilita de realizar consultas e atualizações no sistema.

Como o Google Sheets receberá os dados?

O Google Sheets vai receber os dados através da API integrada com Python. O sistema pega as informações do banco e envia automaticamente para a planilha.

O que acontece se houver falha na comunicação?

Se houver falha na comunicação, os dados podem não chegar no banco ou na planilha. Por isso foi necessário colocar tratamento de erros e testes de conexão para evitar perda de informações.

Como garantir a organização dos dados?

A organização dos dados foi garantida criando tabelas separadas, organizando as

colunas corretamente e validando os dados antes de salvar o banco.

O que foi alterado em relação ao planejamento inicial?

Em relação ao planejamento inicial, foi necessário mudar algumas partes da lógica do sistema, melhorar a autenticação da API, reorganizar o banco de dados e corrigir erros na comunicação entre o ESP8266 e o sistema.

CODIGO DA API

```
Follow link (ctrl + click) ts.google.com/feeds",
"https://www.googleapis.com/auth/drive"
]

# =====
# LOGIN GOOGLE
# =====
creds = ServiceAccountCredentials.from_json_keyfile_name(
    projetoarduinotest,
    scope
)
client = gspread.authorize(creds)
# =====
# PLANILHA
# =====
sheet = client.open("TemperaturaESP").sheet1
print("Google Sheets conectado!")
# =====
# HOME
# =====
@app.route('/')
def home():
    return "API Google Sheets funcionando"
# =====
# RECEBER DADOS
# =====
@app.route('/temperatura')
def receber():
    try:
        # =====
        # RECEBER DADOS
        # =====
        sensor = request.args.get('sensor')
        temperatura = request.args.get('temperatura')
        # =====
        # DATA/HORA
        # =====
        agora = datetime.now().strftime("%d/%m/%Y %H:%M:%S")
        # =====
        # TERMINAL
        # =====
        print("=====")
        print("📡 NOVO DADO")
        print("Sensor:", sensor)
        print("Temperatura:", temperatura)
        print("Data/Hora:", agora)
```

Este código da API, foi feito e testado, entretanto não está funcionando para a visualização correta da API, o código está correto, o problema é por parte na nossa execução de código que não está funcionando.